

Supplementary material: Hyperparameter configuration

1 - Multivariate Laplacian Mixture Model

Since the Laplacian mixing technique was developed from scratch based in a Gaussian Mixture variant, we will present in more detail how the algorithm works.

One of the classic machine learning techniques based on Bayesian inference, which has proven quite effective in various applications, is the Gaussian Mixture Model (GMM). We can also derive an analogous approach using a multivariate Laplacian probability density function. Perhaps we can call this variant of the original technique a Laplacian Mixture Model (LMM). The symmetric multivariate Laplace distribution can be expressed as

$$\mathcal{L}(x|\mu_k, \Sigma_k) = \frac{2}{\sqrt{(2\pi)^D |\Sigma_k|}} \left(\frac{(x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)}{2} \right)^{\frac{\vartheta}{2}} \beta_{\vartheta} \left(\sqrt{2(x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)} \right) \quad (1)$$

and

$$\vartheta = \frac{2 - D}{2} \quad (2)$$

β_{ϑ} corresponds to the modified Bessel function of the second type of order ϑ . Each Laplacian probability density function $\mathcal{L}(x|\mu_k, \Sigma_k)$ is a conditional probability model and is called a component of the mixture and has its own mean μ_k , a vector of dimension ($D \times 1$), and its covariance matrix Σ_k of dimension ($D \times D$). $|\Sigma_k|$ corresponds to the determinant of the covariance matrix.

Consider a system of random variables of dimension D where there are N observed instances. This means that there are N measured points in a D -dimensional space. Using the total probability theorem, the probability density can be represented as a linear combination of M Laplacian densities of the form (LMM)

$$p(x) = \sum_{k=1}^M \pi_k \mathcal{L}(x|\mu_k, \Sigma_k) \quad (3)$$

and

$$\sum_{k=1}^M \pi_k = 1, \quad 0 \leq \pi_k \leq 1 \quad (4)$$

where $p(x)$ is called the mixture of Laplacian densities. In its simplest form, in the unsupervised approach, LMM is a clustering algorithm. M corresponds to the number of clusters one wishes to identify. Based on Bayes' theorem, one can write

$$\gamma_k = p(C_k|x) = \frac{p(C_k) p(x|C_k)}{\sum_{k=1}^M p(C_k) p(x|C_k)} = \frac{\pi_k \mathcal{L}(x|\mu_k, \Sigma_k)}{\sum_{k=1}^M \pi_k \mathcal{L}(x|\mu_k, \Sigma_k)} \quad (5)$$

In equation (5), a latent variable γ (gamma) was introduced, which can be calculated for each point in the feature space. In practice, we do not observe the latent variables, but we can estimate them. The LMM approach uses the Expectation Maximization (EM) algorithm. This objective is achieved by maximizing the log of the likelihood function, which is given by

$$\ln p(\mathbf{X}|\pi, \mu, \Sigma) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^M \pi_k \mathcal{L}(x|\mu_k, \Sigma_k) \right\} \quad (6)$$

Each cluster is modeled according to a different Laplacian distribution. This probabilistic approach allows for flexible data modeling, instead of rigid cluster assignments (partitions of the sample space) as in the k-means technique. This means that each point in the D-dimensional Hilbert space (points in the data space to be classified) could be associated with any of the distributions with a corresponding probability. In fact, each distribution has some "responsibility" in generating a point in the specific data space.

The **EM** algorithm consists of two stages, an **E** (Expectation) stage and an **M** (Maximization) stage. Fig. 1 illustrates, using a block diagram, the constituent modules of the algorithm and the information flow. In the first stage (**E** stage), it is necessary to estimate the posterior distribution of our latent variables γ conditional on our weight vector ($\boldsymbol{\pi}$), the mean vector ($\boldsymbol{\mu}$), and also conditional on our covariance matrix ($\boldsymbol{\Sigma}$). The **E** stage estimate requires initialization.

Using the expectation maximization algorithm, we have step **E** of the Laplacian mixture algorithm. In this step, an estimate of the latent variables (posterior probability) is made.

$$\gamma_{kn} = p(C_k | \mathbf{x}_n) = \frac{\boldsymbol{\pi}_k^{(j)} \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k^{(j)}, \boldsymbol{\Sigma}_k^{(j)})}{\sum_{k=1}^M \boldsymbol{\pi}_k^{(j)} \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k^{(j)}, \boldsymbol{\Sigma}_k^{(j)})} \quad (7)$$

After calculating the posterior probability, it is necessary to update the model parameters, maximizing the log of the likelihood function. Equations (8), (9) and (10) correspond to performing step **M**.

$$\boldsymbol{\mu}_k^{(j+1)} = \frac{\sum_{n=1}^N \gamma_{kn} \mathbf{x}_n}{\sum_{n=1}^N \gamma_{kn}} \quad (8)$$

$$\boldsymbol{\Sigma}_k^{(j+1)} = \frac{\sum_{n=1}^N \gamma_{kn} (\mathbf{x}_n - \boldsymbol{\mu}_k^{(j+1)})^T (\mathbf{x}_n - \boldsymbol{\mu}_k^{(j+1)})}{\sum_{n=1}^N \gamma_{kn}} \quad (9)$$

$$\boldsymbol{\pi}_k^{(j+1)} = \frac{1}{N} \sum_{n=1}^N \gamma_{kn} \quad (10)$$

These two steps are repeated until convergence. γ_{kn} is called the "latent variable," also known as the "cluster responsibility," and can be used to classify other instances after training (measured data).

$$k = \max\{\gamma_{kn}\} = \max\{p(C_k | \mathbf{x}_n)\} \quad (11)$$

Malahanobis distance can also be used as a classification metric.

$$k = \max\{\gamma_{kn}\} = \max\{p(C_k | \mathbf{x}_n)\} \quad (12)$$

$$k = \min\{d_k(\mathbf{x}_n)\} \quad (13)$$

and

$$d_k(\mathbf{x}_n) = \sqrt{(\mathbf{x}_n - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x}_n - \boldsymbol{\mu}_k)} \quad (14)$$

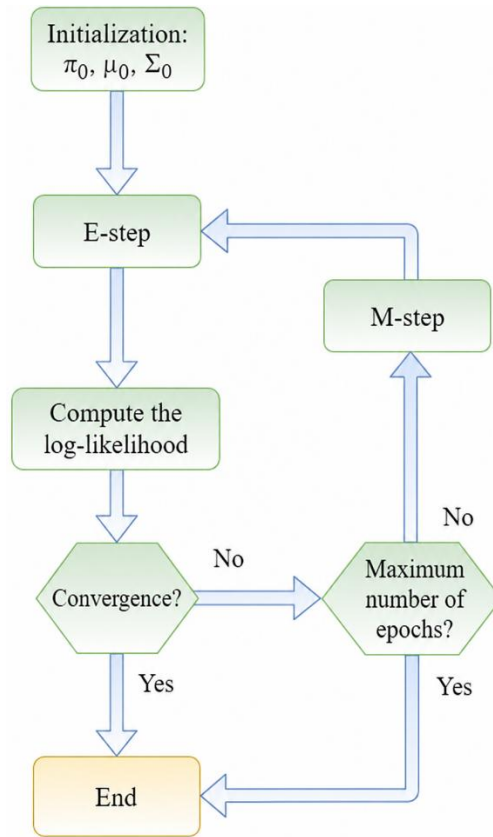


Fig. 1. Block diagram of a probability density mixture model algorithm

Table 1. Laplacian Mixture Model

Hyperparameter	Description	value
n_components	Number of mixture components.	1
tol	Convergence threshold for the EM algorithm.	0.0001
grad_inc	Stopping criterion based on the increment of the log likelihood.	1e-7
max_iter	Maximum number of EM iterations.	1000
init_params	Method used to initialize parameters.	'kmeans'
weights_init	User-defined initial mixture weights.	None
means_init	User-defined initial component means.	None
verbose	Verbosity level.	0

Table 2. Logistic Regression (LR)

Hyperparameter	Description	Value
penalty	Regularization type	l2
dual	Dual or primal formulation	False
tol	Optimization stopping tolerance	1e-5
C	Inverse regularization strength	1.0
fit_intercept	Include intercept term	True
intercept_scaling	Intercept scaling (liblinear only)	1
class_weight	Class weighting	None
random_state	Random seed	None
solver	Optimization algorithm	lbfgs
max_iter	Maximum iterations	100
multi_class	Multiclass strategy	auto
verbose	Verbosity level	0
warm_start	Reuse previous solution	False
n_jobs	Parallel processing	None
l1_ratio	Elastic-Net mixing parameter	None

Table 3. Linear Discriminant Analysis (LDA)

Hyperparameter	Description	Value
solver	Solver algorithm	svd
shrinkage	Covariance shrinkage	None
priors	Class prior probabilities	None
n_components	Number of discriminant components	None
store_covariance	Store covariance matrix	False
tol	Rank estimation threshold	1e-4
covariance_estimator	Covariance estimator object	None

Table 4. Quadratic Discriminant Analysis (QDA)

Hyperparameter	Description	Value
priors	Class prior probabilities	None
reg_param	Covariance regularization	0.0
store_covariance	Store covariance matrices	False
tol	Rank-deficiency threshold	1e-4

Table 5. Decision Tree (DT)

Hyperparameter	Description	Value
criterion	Split quality criterion	gini
splitter	Split selection strategy	best
max_depth	Maximum tree depth	None
min_samples_split	Minimum samples to split a node	2
min_samples_leaf	Minimum samples in a leaf	1
min_weight_fraction_leaf	Minimum weighted fraction in a leaf	0.0
max_features	Number of features considered at split	None
random_state	Random seed	None
max_leaf_nodes	Maximum number of leaves	None
min_impurity_decrease	Minimum impurity decrease	0.0
class_weight	Class weighting	None
ccp_alpha	Cost-complexity pruning parameter	0.0
monotonic_cst	Monotonic constraints	None

Table 6. Gaussian Naïve Bayes (GNB)

Hyperparameter	Description	Value
priors	Class prior probabilities	None
var_smoothing	Portion of largest variance added to variances for stability	1e-9

Table 7. Gaussian Mixture Model

Hyperparameter	Description	value
n_components	Number of mixture components.	1
covariance_type	Type of covariance matrix used by each component.	'full'
tol	Convergence threshold for the EM algorithm.	0.001
reg_covar	Regularization added to covariance diagonals for numerical stability.	1e-7
max_iter	Maximum number of EM iterations.	120
n_init	Number of initializations performed.	1
init_params	Method used to initialize parameters.	'kmeans'
weights_init	User-defined initial mixture weights.	None
means_init	User-defined initial component means.	None
precisions_init	User-defined initial precision matrices.	None
random_state	Controls random initialization.	None
warm_start	Reuses previous solution for subsequent fitting.	False
verbose	Verbosity level.	0
verbose_interval	Interval for printing verbose information.	10

Table 8. AdaBoost Classifier

Hyperparameter	Description	value
estimator	Base classifier used in the boosting ensemble. If None, a decision stump is used.	None
n_estimators	Maximum number of weak learners.	55
learning_rate	Weight applied to each classifier contribution.	1.0
random_state	Controls random seed generation.	None

Note: when estimator=None, the default base estimator is DecisionTreeClassifier(max_depth=1).

Table 9. Bagging Classifier

Hyperparameter	Description	value
estimator	Base estimator trained on random subsets of the data.	None
n_estimators	Number of base estimators in the ensemble.	12
max_samples	Number or proportion of samples drawn for each base estimator.	None
max_features	Number or proportion of features drawn for each base estimator.	1.0
bootstrap	Whether samples are drawn with replacement.	True
bootstrap_features	Whether features are drawn with replacement.	False
oob_score	Whether to estimate generalization error using out-of-bag samples.	False
warm_start	Reuses previous fitted ensemble and adds more estimators.	False
n_jobs	Number of parallel jobs.	None
random_state	Controls random resampling.	None
verbose	Verbosity level.	0

Note: when estimator=None, the default base estimator is DecisionTreeClassifier.

Table 10. Extra-Trees Classifier

Hyperparameter	Description	value
n_estimators	Number of trees in the forest.	150
criterion	Function used to measure split quality.	'gini'
max_depth	Maximum depth of each tree.	None
min_samples_split	Minimum number of samples required to split an internal node.	3
min_samples_leaf	Minimum number of samples required at a leaf node.	1
min_weight_fraction_leaf	Minimum weighted fraction required at a leaf node.	0.0
max_features	Number of features considered at each split.	'sqrt'
max_leaf_nodes	Maximum number of leaf nodes.	None
min_impurity_decrease	Minimum impurity decrease required for a split.	0.0
bootstrap	Whether bootstrap samples are used.	False
oob_score	Whether out-of-bag samples are used to estimate performance.	False
n_jobs	Number of parallel jobs.	None
random_state	Controls randomness of tree construction.	None
verbose	Verbosity level.	0
warm_start	Reuses previous fitted ensemble.	False
class_weight	Class weighting strategy.	None
ccp_alpha	Complexity parameter for cost-complexity pruning.	0.0
max_samples	Number or fraction of samples drawn when bootstrap=True.	None
monotonic_cst	Monotonic constraints for features.	None

Table 11. Ensemble Extra-Tree

Hyperparameter	Description	value
criterion	Function used to measure split quality.	'gini'
splitter	Strategy used to choose each split.	'random'
max_depth	Maximum depth of the tree.	None
min_samples_split	Minimum number of samples required to split an internal node.	2
min_samples_leaf	Minimum number of samples required at a leaf node.	1
min_weight_fraction_leaf	Minimum weighted fraction required at a leaf node.	0.0
max_features	Number of features considered when looking for the best split.	'sqrt'
random_state	Controls randomness of the split selection.	None
max_leaf_nodes	Maximum number of leaf nodes.	None
min_impurity_decrease	Minimum impurity decrease required for a split.	0.0
class_weight	Class weighting strategy.	None
ccp_alpha	Complexity parameter for cost-complexity pruning.	0.0
monotonic_cst	Monotonic constraints for features.	None

Table 12. Gradient Boosting Classifier

Hyperparameter	Description	value
loss	Loss function optimized during boosting.	'log_loss'
learning_rate	Shrinks the contribution of each tree.	0.1
n_estimators	Number of boosting stages.	120
subsample	Fraction of samples used to fit each base learner.	1.0
criterion	Split quality measure; deprecated in current scikit-learn.	'deprecated'
min_samples_split	Minimum samples required to split an internal node.	2
min_samples_leaf	Minimum samples required at a leaf node.	1
min_weight_fraction_leaf	Minimum weighted fraction required at a leaf node.	0.0
max_depth	Maximum depth of individual regression trees.	3
min_impurity_decrease	Minimum impurity decrease required for a split.	0.0
init	Initial estimator used before boosting.	None
random_state	Controls random seed.	None
max_features	Number of features considered at each split.	None
verbose	Verbosity level.	0
max_leaf_nodes	Maximum number of leaf nodes.	None
warm_start	Reuses previous solution and adds more estimators.	False
validation_fraction	Fraction of training data used for early stopping validation.	0.1
n_iter_no_change	Number of iterations with no improvement before early stopping.	None
tol	Tolerance for early stopping.	0.0001
ccp_alpha	Complexity parameter for cost-complexity pruning.	0.0

Table 13. Random Forest Classifier

Hyperparameter	Description	value
n_estimators	Number of trees in the forest.	100
criterion	Function used to measure split quality.	'gini'
max_depth	Maximum depth of each tree.	None
min_samples_split	Minimum samples required to split an internal node.	2
min_samples_leaf	Minimum samples required at a leaf node.	1
min_weight_fraction_leaf	Minimum weighted fraction required at a leaf node.	0.0
max_features	Number of features considered at each split.	'sqrt'
max_leaf_nodes	Maximum number of leaf nodes.	None
min_impurity_decrease	Minimum impurity decrease required for a split.	0.0
bootstrap	Whether bootstrap samples are used.	True
oob_score	Whether out-of-bag samples are used to estimate generalization performance.	False
n_jobs	Number of parallel jobs.	None
random_state	Controls randomness of bootstrap sampling and feature selection.	None
verbose	Verbosity level.	0
warm_start	Reuses previous fitted forest and adds more trees.	False
class_weight	Class weighting strategy.	None
ccp_alpha	Complexity parameter for cost-complexity pruning.	0.0
max_samples	Number or fraction of samples drawn when bootstrap=True.	None
monotonic_cst	Monotonic constraints for features.	None

Table 14. Histogram Gradient Boosting Classifier

Hyperparameter	Description	value
loss	Loss function used in the boosting process.	'log_loss'
learning_rate	Shrinkage factor applied to leaf values.	0.1
max_iter	Maximum number of boosting iterations.	150
max_leaf_nodes	Maximum number of leaves per tree.	31
max_depth	Maximum depth of each tree.	None
min_samples_leaf	Minimum number of samples per leaf.	22
l2_regularization	L2 regularization applied to leaf values.	0.0
max_features	Proportion of features randomly selected at each split.	1.0
max_bins	Maximum number of bins for continuous features.	255
categorical_features	Indicates which features are categorical.	'from_dtype'
monotonic_cst	Monotonic constraints for features.	None
interaction_cst	Interaction constraints among features.	None
warm_start	Reuses previous solution for additional fitting.	False
early_stopping	Early stopping strategy.	'auto'
scoring	Scoring criterion for early stopping.	'loss'
validation_fraction	Fraction of training data used for validation.	0.1
n_iter_no_change	Number of iterations without improvement before stopping.	10
tol	Tolerance for early stopping.	5e-8
verbose	Verbosity level.	0
random_state	Controls randomness.	None
class_weight	Class weighting strategy.	None

Table 15. XGBoost Classifier

Hyperparameter	Description	value
booster	Type of boosting model.	'gbtree'
device	Device used for computation.	'cpu'
verbosity	Controls message printing.	1
validate_parameters	Checks whether input parameters are valid.	True in Python
nthread	Number of parallel threads.	Maximum available if not set
objective	Learning objective. For classifier wrapper, commonly set according to classification task.	'binary:logistic' for binary classification
n_estimators	Number of boosting rounds in the scikit-learn API.	120
learning_rate / eta	Step-size shrinkage used to reduce overfitting.	0.3
gamma / min_split_loss	Minimum loss reduction required for further partitioning.	0
max_depth	Maximum depth of each tree.	7
min_child_weight	Minimum sum of instance weight required in a child.	1
max_delta_step	Maximum leaf weight update step.	0
subsample	Fraction of training samples used per boosting iteration.	1
sampling_method	Method used to sample training instances.	'uniform'
colsample_bytree	Fraction of features sampled per tree.	1
colsample_bylevel	Fraction of features sampled per tree level.	1
colsample_bynode	Fraction of features sampled per split node.	1
reg_lambda / lambda	L2 regularization term.	1
reg_alpha / alpha	L1 regularization term.	0
tree_method	Tree construction algorithm.	'auto'
grow_policy	Policy controlling how new nodes are added.	'depthwise'
max_leaves	Maximum number of leaves.	0
max_bin	Maximum number of histogram bins.	256
num_parallel_tree	Number of parallel trees per boosting iteration.	1
multi_strategy	Strategy for multi-output or multiclass training.	'one_output_per_tree'
eval_metric	Evaluation metric.	Automatically selected according to objective
random_state	Random seed in the scikit-learn API.	None